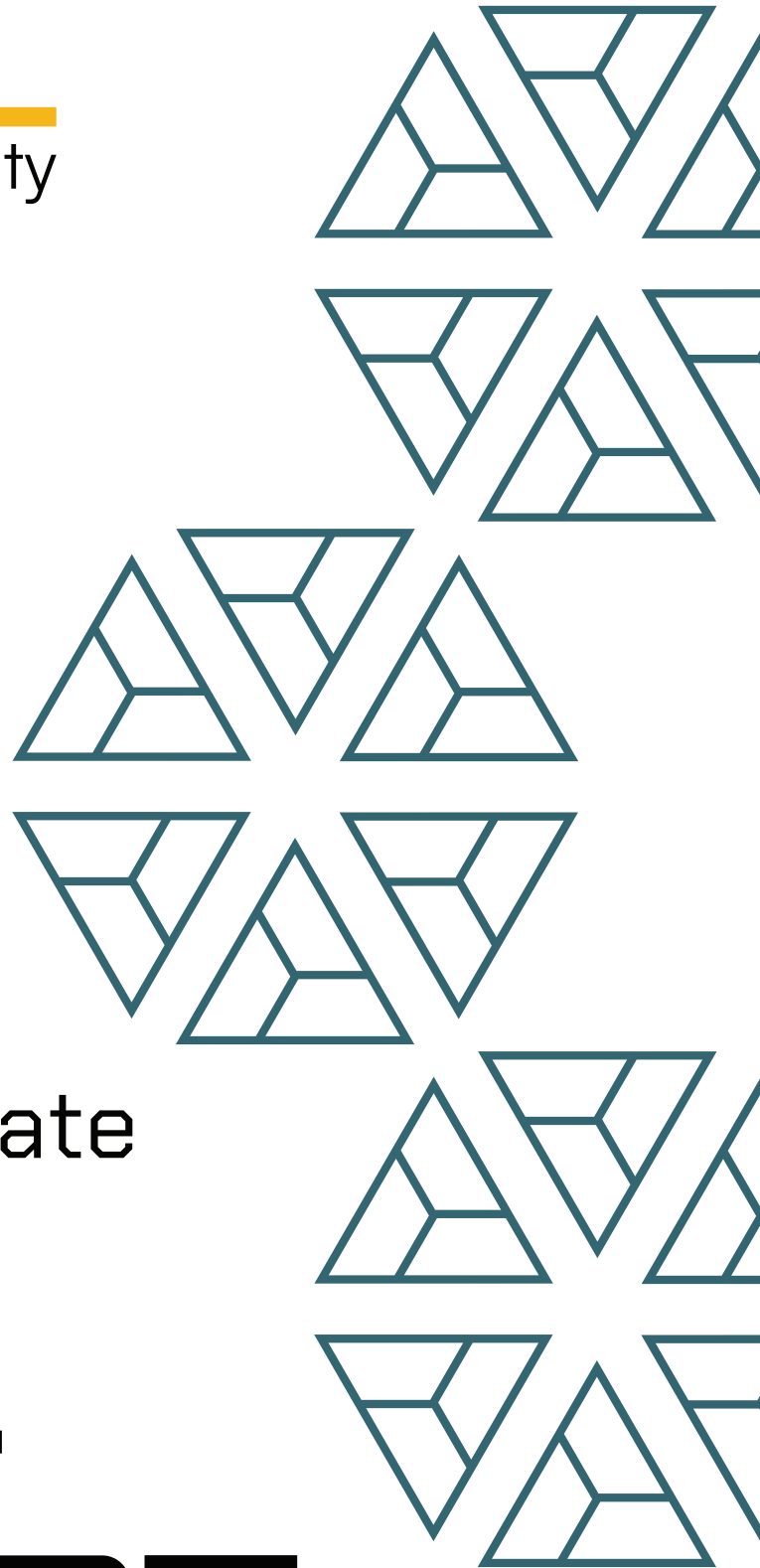




BAIL
security



Symbiotic
Rewards V2 Update

FINAL REPORT

April '2026

Disclaimer:

Security assessment projects are time-boxed and often reliant on information that may be provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

The content of this assessment is not an investment. The information provided in this report is for general informational purposes only and is not intended as investment, legal, financial, regulatory, or tax advice. The report is based on a limited review of the materials and documentation provided at the time of the audit, and the audit results may not be complete or identify all possible vulnerabilities or issues. The audit is provided on an "as-is," "where-is," and "as-available" basis, and the use of blockchain technology is subject to unknown risks and flaws.

The audit does not constitute an endorsement of any particular project or team, and we make no warranties, expressed or implied, regarding the accuracy, reliability, completeness, or availability of the report, its content, or any associated services or products. We disclaim all warranties, including the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

We assume no responsibility for any product or service advertised or offered by a third party through the report, any open-source or third-party software, code, libraries, materials, or information linked to, called by, referenced by, or accessible through the report, its content, and the related services and products. We will not be liable for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract.

The contract owner is responsible for making their own decisions based on the audit report and should seek additional professional advice if needed. The audit firm or individual assumes no liability for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract. The contract owner agrees to indemnify and hold harmless the audit firm or individual from any and all claims, damages, expenses, or liabilities arising from the use or reliance on the audit report or the smart contract.

By engaging in a smart contract audit, the contract owner acknowledges and agrees to the terms of this disclaimer.

1. Project Details

Important:

Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Symbiotic - RewardsV2 Update - Audit Report
Website	symbiotic.fi
Language	Solidity
Methods	Manual Analysis
Github repository	Last audited commit: https://github.com/symbioticfi/rewards-v2/tree/15762166b8f96bef1c1ddec6ceb458208b61c668/src/contracts New commit: https://github.com/symbioticfi/rewards-v2-mirror/tree/2523bef4ff6c38c16f5e10500d3cb4b0bd73ec24/src Differential Audit: FeeRegistry: https://www.diffchecker.com/PufREXhV/ ProtocolFees: https://www.diffchecker.com/Hk50y8Yf/ Rewards: https://www.diffchecker.com/MSjYMBdQ/ VaultSnapshotRewards: https://www.diffchecker.com/JCLXNjLB/ Full Audit: DonationRewards CuratorFees
Resolution 1	https://github.com/symbioticfi/rewards-v2-mirror/tree/ced075ef3b1b65dd363183dfc3e920fe285a5df0

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no changes made)	Failed resolution	Open
High	1	1				
Medium	2	1		1		
Low	4	1		3		
Informational	15	1		14		
Governance	1			1		
Total	23	4		19		

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

FeeRegistry

The following changes have been made to the **FeeRegistry** contract:

- > new mapping for instant withdraw fee
- > initializer disabled
- > non-semantic changes to include adapter
- > helper function for instant withdraw fee

Diff: <https://www.diffchecker.com/PufREXhV/>

Please note that issues which have previously been acknowledged, will not be raised again.

Issue_01	Race condition from instantWithdrawFee change
Severity	Low
Description	Currently, the instantWithdrawFee can be changed by the vault curator which directly influences instant withdrawals within the VaultV2 contract [out of scope]. Such an increase can result in a race condition where users withdraw and pay an unexpectedly large fee. This is valid for all fees in general as well.
Recommendations	Consider either implementing a slippage check in the VaultV2 contract or acknowledging this issue.
Comments / Resolution	Acknowledged

ProtocolFees

The following changes have been made to the **ProtocolFees** contract:

> non-semantic changes to include adapter {networkOrAdapter}

Diff: <https://www.diffchecker.com/Hk50y8Yf/>

Please note that issues which have previously been acknowledged, will not be raised again.

No issues have been observed from these changes.

Rewards

The following changes have been made to the **Rewards** contract:

- > inherit DonationRewards
- > expose multicall direct w/o inheriting
- > expose new donation reward type

Diff: <https://www.diffchecker.com/MSjYMBdQ/>

Please note that issues which have previously been acknowledged, will not be raised again.

No issues have been observed from these changes.

VaultSnapshotRewards

Disclaimer: Please note that issues which have been acknowledged in the previous scope are not listed here. This section only covers new issues that arise from the changes described below.

Important note: During the resolution round, this contract is treated “as newly deployed” and there will be no upgrade assumptions. Historic rewards are disregarded here.

The following changes have been made to the [VaultSnapshotRewards](#) contract:

- > Inherits CuratorFees which inherits ProtocolFees
- > outsource mapping for curatorFees
- > introduce `_totalOperatorNetworkSharesCache`
 - > used for gas optimization `NETWORK_RESTAKE`
- > introduce `_subnetworkFilledCache`
- > introduce `_activeWithdrawalSharesCache`
 - > used as divisor during claim
- > `distributeVaultSnapshotRewards` refactored
 - > donation support
 - > support for V2 [withdrawalShares]
 - > handling V1 timestamps for VaultV2
 - > caching shares for `NETWORK_RESTAKE`
 - > handling the `UNIVERSAL` delegator
 - > fee subtraction slightly changed
 - > allocate rewards for V2
 - > allocate rewards for V1
 - > allocate rewards for V2 before migration
 - > donation
- > `claimVaultSnapshotRewards` refactored
 - > claiming for V1
 - > claiming for V2
 - > claiming for V2 pre migration
- > `claimOperatorFees` refactoring
 - > conditional check if there are fees at all
 - > using cached value for `NETWORK_RESTAKE` instead of fetching new [gas]
 - > else if condition for `OPERATOR_NETWORK_SPECIFIC` [explicit]
 - > handling for `UNIVERSAL` [proportional allocated <-> filledAt]
- > curator fee claiming removed [outsourced to inherited contract]

Diff: <https://www.diffchecker.com/JCLXNjLB/>

Disclaimer: Please note the highlighted invariants with regards to the UNIVERSAL delegator. These have not been validated by Bailsec because this contract is out of scope. Potential edge-cases to this mechanism need to be investigated independently.

Core Invariants:

INV 1: Rewards are tied to vault/network/token

INV 2: Rewards can only be distributed to valid networks

INV 3: OperatorFees are ignored if:

- NETWORK_RESTAKE + no shares
- FULL_RESTAKE
- UNIVERSAL + index=0 OR filled = 0

INV 4: Rewards for a migrated Vault which are supposed to be before the migration time should go to the oldDelegator

INV5: FULL_RESTAKE / NETWORK_RESTAKE / OPERATOR_SPECIFIC / OPERATOR_NETWORK_SPECIFIC are old delegators

INV 6: In VaultV2 distributions, amountToDeposits and amountToWithdrawals is allocated proportional on its part of activeSharesAt and activeWithdrawalsAt

INV 7: amountToDeposit and amountToWithdrawals must be calculated based on distributed amount after protocol/curator/operator fees

INV 8: Distribution donations are only allowed for VaultV2 contracts

INV 9: VaultV2.activeSharesAt / VaultV2.activeWithdrawalSharesAt / VaultV2.activeWithdrawalsAt / VaultV2.activeStakeAt / VaultV2.activeSharesOfAt / VaultV2.activeWithdrawalSharesOfAt can only be called with past block.timestamp and must always return immutable values.

INV 10: IUniversalDelegator[delegator].getSlotOfNetworkAt[subnetwork, timestamp] and IUniversalDelegator[delegator].getFilledAt[subnetworkIndex, 0, timestamp] must always be immutable for past timestamps

INV 11: The aggregated return value of `getAllocatedAt` for all operators must match the return value of `getFilledAt`

Please note that issues which have previously been acknowledged, will not be raised again.

Issue_02	Storage collision due to extension of struct data within array
Severity	High
Description	<code>VaultSnapshotRewardsStorage</code> stores all historical snapshot rewards inside <code>_rewards</code>, which is a nested mapping whose value is <code>RewardDistribution[]</code>. In the on-chain version, each <code>RewardDistribution</code> element occupies four storage slots: one packed slot for <code>subnetworkId</code> and <code>delegator</code>, one packed slot for <code>delegatorType</code> and timestamp, one slot for amount, and one slot for <code>operatorsFees</code>. In the upgraded version, the contract keeps the same <code>_rewards</code> array in the same storage namespace, but extends the struct by renaming amount to <code>amountToDeposits</code> and appending a new <code>amountToWithdrawals</code> field. This changes the storage size of each array element from four slots to five slots while preserving the same storage root for the array. Solidity stores dynamic arrays of structs using a fixed element stride. Historical <code>_rewards</code> entries were written using the old stride of four slots, but after the upgrade the new implementation reads them using a stride of five slots. As a result, historical rewards are decoded from incorrect offsets. Concretely, if a historical reward stream had at least two entries, old entry 0 occupied slots base + 0 through base + 3, while old entry 1 occupied slots base + 4 through base + 7. After the upgrade, the new implementation reads entry 0 from base + 0 through base + 4, so the newly added <code>amountToWithdrawals</code> field is read from base + 4, which is actually the first packed slot of the next historical entry. Then the upgraded implementation reads entry 1 starting from base + 5, which means every field of entry 1 and later entries is shifted and corrupted. The root cause is that the upgrade preserves the same <code>_rewards</code> storage root while changing the size of the struct used inside the dynamic array, which is not storage-compatible in place.

	There are several different impacts from this but alone the description above is sufficient to highlight the critical nature of this issue.
Recommendations	Consider refactoring the way how <code>amountToWithdrawal</code> is written/read. If there are no entries at the time of upgrade, a safe upgrade can be made by validating that such a state is true.
Comments / Resolution	Resolved, it will be ensured that there are no entries.

Issue_03	Historic rewards for NETWORK_RESTAKE are unclaimable
Severity	Medium
Description	Within the new version, whenever rewards for the NETWORK_RESTAKE operator are distributed, it will cache the total shares as follows: <pre><i>_vaultSnapshotRewardsStorage[]_totalOperatorNetworkSharesCache[vault][subnetwork][timestamp] = totalOperatorNetworkShares;</i></pre> During claiming, it will then fetch it: <pre><i>.mulDiv(reward.operatorsFees, _vaultSnapshotRewardsStorage[_totalOperatorNetworkSharesCache[vault]][network.subnetwork(reward.subnetworkId)][reward.timestamp]);</i></pre> This will not work with historic rewards, as the previous version didn't cache the shares. As such, there is no valid entry in the mapping and operators cannot claim their fees. One thing is interesting to note: It is theoretically possible to distribute rewards for this specific timestamp, which will then populate the _totalOperatorNetworkSharesCache with the correct value and allows for claiming historic rewards.
Recommendations	Consider providing an alternative route to claim historic fees.
Comments / Resolution	Acknowledged, it will be ensured that there are no historical entries.

Issue_04	Edge-case within VaultV2 state can result in distributable but locked rewards
Severity	Medium
Description	The VaultV2 contract exposes shares and underlying amounts and exposes a donation mechanism which increases the amounts but keeps the shares steady. This can result in a case where $\text{activeStake} > 0$ and $\text{shares} = 0$: 1. Alice deposits 100 collateral into VaultV2. - $\text{activeStake} = 100$ - $\text{activeShares} = 100$ 2. A donation of 50 collateral is sent through the rewards flow into <code>VaultV2.donate()</code>. - $\text{activeStake} = 150$ - $\text{activeShares} = 100$ 3. Alice redeems all 100 active shares. - $\text{withdrawnAssets} = \text{floor}(100 * (150 + 1) / (100 + 1)) = \text{floor}(15100 / 101) = 149$ 4. Post-redeem: - $\text{activeStake} = 1$ - $\text{activeShares} = 0$ Alice will also own <code>withdrawalShares</code>. If now rewards are distributed in this scenario, the distribution does not revert because <code>activeWithdrawalShares > 0</code>: <pre> if (activeShares == 0 && activeWithdrawalShares == 0) { revert InvalidRewardTimestamp(); } </pre> If these rewards are attempted to be claimed, it will revert due to division by zero:

	<pre> if {amountToDeposits > 0} { amount += IVaultV2(vault).activeSharesOfAt(msg.sender, timestamp, activeSharesHints[i]) .mulDiv(amountToDeposits, _vaultSnapshotRewardsStorage[]._activeSharesCache[vault][timest amp]); } </pre> This will likewise lock rewards for activeStakes and activeWithdrawals.
Recommendations	In our opinion, it should be ensured that the vault cannot enter in such an invalid state. It is likely that this state will have direct effects on the vault integrity. However, the VaultV2 contract is not in our scope. It should also be checked whether it is possible that activeWithdrawalShares = 0 and amountToWithdrawal > 0. This would be part of the VaultV2 review.
Comments / Resolution	Resolved.

Issue_05	Migration to V2 can result in undistributable rewards which would have been distributable under V1
Severity	Low
Description	VaultV1 and VaultV2 use different accounting models for snapshot reward settlement. In VaultV1, slashing reduces <code>_activeStake</code> but does not burn <code>_activeShares</code>, so a fully slashed timestamp can still have <code>activeSharesAt(timestamp) > 0</code> while <code>activeStakeAt(timestamp) = 0</code>. The old <code>VaultSnapshotRewards</code> implementation for V1 only validated that <code>activeSharesAt(timestamp)</code> was non-zero which is indeed true in the above described example and then distributed the entire external reward amount to depositors pro rata by shares, without performing any deposit/withdrawal asset split. After upgrade, the new <code>VaultSnapshotRewards</code> implementation supports VaultV2 semantics and decides whether to use the V2 settlement path by checking the vault's current version, not whether the rewarded timestamp belongs to the pre-migration V1 era. An issue occurs when rewards are distributed after a vault has migrated to <code>VaultV2</code>, but the chosen reward timestamp is still a pre-migration VaultV1 timestamp that was fully slashed. In that situation, the historical V1 checkpoints can still report <code>activeSharesAt(timestamp) > 0</code>, because V1 did not burn shares during slashing, while <code>activeStakeAt(timestamp) = 0</code>. As above explained, rewards were still distributed as long as it was for V1. The new <code>distributeVaultSnapshotRewards</code> function first performs a share-based validity check and accepts the timestamp because <code>activeSharesAt(timestamp)</code> is non-zero. It then selects the V2 settlement branch solely because the vault's

	current version is V2. In that V2 branch, it computes the withdrawal-side allocation using <code>activeWithdrawalsAt(timestamp)</code> and <code>activeStakeAt(timestamp) + activeWithdrawalsAt(timestamp)</code> as the denominator. For a pre-migration fully slashed timestamp with no withdrawals, both values are zero, so the denominator is zero and the distribution reverts. Under the old V1 rewarder, the same timestamp would have remained distributable because the V1 path did not attempt any asset-side split.
Recommendations	We recommend acknowledging this issue because this is only a very rare edge-cases which is likely not happening in practice and a fix for this issue tends to introduce new side-effects which could remain undetected and from higher severity.
Comments / Resolution	Acknowledged.

Issue_06	Usage of <code>activeSharesHint</code> for <code>activeStakeAt</code> is inaccurate
Severity	Low
Description	During reward distributions it uses <code>activeSharesHint</code> for the <code>activeStakeAt</code> function. This is intuitively correct but donation pushes to <code>activeStake</code> and not to <code>activeShares</code> which then makes this a valid issue, as it creates an inconsistency. Essentially, After any <code>donate</code> or <code>onSlash</code> call, <code>_activeStake</code> accumulates extra checkpoint entries that <code>_activeShares</code> does not have. The checkpoint indices between the two traces become permanently shifted.
Recommendations	Consider using different hints.
Comments / Resolution	Resolved.

Issue_07	<code>timestamp</code> parameter within <code>distributeVaultSnapshotRewards</code> allows for ambiguous behavior in case of donation
Severity	Low
Description	If a donation is executed, the timestamp parameter does not define for which time rewards are allocated, as these are tied to existing stakes. However, it is still used for purposes such as the delegator determination and for fee determination, allowing a user to pass a beneficial timestamp. This issue is only marked as low because callers are expected to be system contracts from Symbiotic.
Recommendations	This issue is harder to fix as there cannot be a generalized rule for when a timestamp should be during a donation. One idea could be to make it just a few seconds behind the current <code>block.timestamp</code> , to prevent staleness. Potential side-effects from such a solution must be kept in mind.
Comments / Resolution	Acknowledged.

Issue_08	Unconventional calculation of fees
Severity	Informational
Description	Within the distributeVaultSnapshotRewards function, the distribution fee is treated as follows: - deduct protocolFee from it - calculate operatorFee based on (distributionFee - protocolFee) - calculate curatorFee based on (distributionFee - protocolFee) While this works as long as operator and curator fees are capped with 50%, it can become problematic in case they are ever set > 50%.
Recommendations	Consider keeping this in mind.
Comments / Resolution	Acknowledged.

Issue_09	Generalized IVaultV2 interface usage for VaultV1
Severity	Informational
Description	Currently, the IVaultV2 is used on multiple occasions where the address is actually the Vault contract. While this is technically incorrect, it does not appear to expose a problem because the commingled selectors are equal. This is for example happening here: <pre>uint256 activeShares = IVaultV2[vault].activeSharesAt(timestamp, distributeVaultSnapshotRewardsHints.activeSharesHint);</pre> and in other spots.
Recommendations	Consider carefully evaluating all spots where this is true and making sure all external calls succeed.
Comments / Resolution	Acknowledged.

Issue_10	Redundant operation whenever active shares are zero
Severity	Informational
Description	Within distributeVaultSnapshotRewards, it checks if active shares are zero and then attempts to cache them: <pre>else if [_vaultSnapshotRewardsStorage()_activeSharesCache[vault][timestamp] == 0]</pre> Since it is a valid state where active shares are zero and active withdrawal shares are non-zero, it will re-enter this branch whenever rewards for such a state are being distributed, writing the same immutable values to the mappings: <pre>_vaultSnapshotRewardsStorage()_activeSharesCache[vault][timestamp] = activeShares;</pre> <pre>_vaultSnapshotRewardsStorage()_activeWithdrawalSharesCache[vault][timestamp] = activeWithdrawalShares;</pre>
Recommendations	Consider acknowledging this issue. Within the normal business logic it will likely be more gas-friendly to only check for one value during every single distribution compared to having rare duplicate SSTOREs.
Comments / Resolution	Acknowledged.

Issue_11	General truncation occasions will have second-order effects
Severity	Informational
Description	Within the contract, there are multiple divisions which result in truncations that have secondary effects such as stranded dust rewards or truncated fees.
Recommendations	Consider acknowledging this issue
Comments / Resolution	Acknowledged.

Issue_12	Incorrect event emission in case of donation
Severity	Informational
Description	If a donation reward happens, the event is emitted as: <pre><i>emit DistributeVaultSnapshotRewards(network, token, vault, subnetwork.identifier(), timestamp, distributionAmount, operatorsFees);</i></pre> The token is actually address[0] in the donation control-flow.
Recommendations	Consider adjusting token parameter if the control-flow is for donations.
Comments / Resolution	Acknowledged.

Issue_13	Donations are not dripped in VaultV2 [OOS]
Severity	Informational
Description	The VaultV2 contract receives rewards as donations which immediately increases the share price instead of being dripped over a period. While this is partially mitigated due to the instant withdrawal fee, it can still become profitable depending on the size of the donation.
Recommendations	Consider re-elaborating the design of the VaultV2 or acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_14	_subnetworkFilledCache experiences redundant duplicate writes
Severity	Informational
Description	Whenever rewards are distributed and the delegator is from type UNIVERSAL, the following snippet will be triggered: <pre> else if {delegatorType == uint64[DelegatorType.UNIVERSAL]} { uint96 subnetworkIndex = lUniversalDelegator[delegator].getSlotOfNetworkAt[subnetwork, timestamp]; uint256 subnetworkFilled = lUniversalDelegator[delegator].getFilledAt[subnetworkIndex, 0, timestamp]; if {subnetworkIndex == 0 subnetworkFilled == 0} { operatorsFees = 0; } else { _vaultSnapshotRewardsStorage[_subnetworkFilledCache[vault][_ subnetwork]][timestamp] = subnetworkFilled; } </pre> As we can see, it will always write to the _subnetworkFilledCache mapping if rewards for the same context are distributed. This is however not problematic as this value is guaranteed to be immutable.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_15	Inconsistency in fee mechanism between donations via <code>VaultRewardSnapshot</code> and <code>DonationRewards</code>
Severity	Informational
Description	Both contracts allow for donating to the <code>VaultV2</code> contract but the <code>VaultSnapshotRewards</code> contract exposes an additional operator fee which is not taken on the <code>DonationRewards</code> contract.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_16	Implicit else branch within <code>distributeVaultSnapshotRewards</code> always expects UNIVERSAL delegator
Severity	Informational
Description	Within the above mentioned function, the else branch is inherently tied to the UNIVERSAL delegator. While that does not seem to expose any issue in the current context, it might become problematic in the future.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_17	Potential fee only reward creation which would not be allowed under V1 vaults
Severity	Informational
Description	VaultV1 did not have an <code>activeWithdrawalSharesAt</code> concept. Its snapshot reward logic only validated <code>activeSharesAt(timestamp)</code> and rejected timestamps with no active shares. During VaultV1 -> VaultV2 migration, <code>VaultV2._migrate</code> seeds <code>_withdrawalSharesCumulative</code> at <code>migrateNextEpochTimestamp</code> and, separately, at <code>migrateTimestamp + epochDuration</code> in order to project still-active V1 withdrawals into the new V2 cumulative-share model. As a result, for timestamps in the current epoch before migration, specifically $T \in [\text{migrateNextEpochTimestamp} - \text{epochDuration}, \text{migrateTimestamp}]$, the post-migration V2 view can report a positive <code>activeWithdrawalSharesAt(T)</code> even though <code>activeWithdrawalsAt(T)</code> remains zero, because V2 withdrawal-bucket asset accounting only starts at <code>migrateTimestamp</code>. These are not true historical V1 withdrawal shares, they are synthetic migration-projected values exposed by the new V2 getter. The upgraded <code>distributeVaultSnapshotRewards</code> selects settlement behavior from the vault's current version rather than from the rewarded timestamp. Therefore, once a vault has migrated to V2, the function uses the V2 validation and settlement path even for pre-migration V1 timestamps. For an old timestamp T in the migration leak window where <code>activeSharesAt(T) == 0</code>, the old V1 implementation would have reverted immediately because V1 only validated active shares. After migration, however, the new implementation evaluates <code>activeWithdrawalSharesAt(T)</code> and can observe a positive synthetic value from <code>_withdrawalSharesCumulative</code> seeding. That causes the new share-based guard to pass even though T

	would have been invalid under V1 semantics. The function then proceeds into the V2 deposit/withdrawal split and computes the denominator from <code>activeStakeAt(T) + activeWithdrawalsAt(T)</code>. For these old timestamps, both values can be zero, so the call reverts later. Additionally, if protocol, curator, and operator fee deductions reduce the user residual to zero while <code>operatorsFees</code> stays positive, the split branch is skipped and the function can still create an operator-only reward entry for a timestamp that the old V1 logic would have rejected. The root cause is the combination of three factors: migration seeding creates synthetic V2 withdrawal-share values for certain pre-migration timestamps; the upgraded rewarder validates timestamp existence on the share side; and settlement is still performed on the asset side using V2 accounting selected from the vault's current version rather than the historical timestamp. If the post-fee user residual becomes zero (due to maximum fees) but <code>operatorsFees</code> remains positive, the upgraded contract can create operator-only rewards for timestamps that the old V1 logic would have rejected, causing operators to receive fees for a time period where no active-share-based reward distribution was supposed to exist.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_18	Changes to function signatures and events could disrupt existing integrations.
Severity	Informational
Description	The following changes have been introduced: 1. A change to a parameter of <code>distributeVaultSnapshotRewards</code> that alters the function signature: - From: <code>DistributeVaultSnapshotRewardsHints calldata hints</code> - To: <code>bytes calldata hints</code> 2. The <code>curatorFees</code> parameter has been removed from the <code>DistributeVaultSnapshotRewards</code> event. These changes could affect integrated parties, such as smart contracts, bots, and scripts, that use the old implementation.
Recommendations	Consider keeping that in mind.
Comments / Resolution	Acknowledged.

DonationRewards

The `DonationRewards` contract is a new contract that will undergo a full audit and is inherited within the Rewards module. It allows any caller to donate rewards in form of the VaultV2's collateral token to any legitimate VaultV2 which triggers notification via `VaultV2.donate` and increases the share price of the underlying vault instantly.

Privileged Functions

- none

Core Invariants:

INV 1: `distributeDonationRewards` can only be called with a vault that is deployed by `VAULT_FACTORY`

INV 2: `distributeDonationRewards` can only be called with a vault that is minimum V2

Issue_19	Curators <code>instantWithdrawalFee</code> influence concerns
Severity	Governance
Description	1. Curators can influence whether the protocol receives fees from instant withdrawals: From the FeeRegistry contract, it is clear that the protocol fee is intended to be controlled only by the contract owner, not by the curator. However, the curator can still influence whether the protocol collects fees from instant withdrawals. When an instant withdrawal fee is charged to the user, that amount is automatically donated to the vault through <code>distributeDonationRewards</code>. During that donation flow, both the protocol and the curator collect a fee, and the remainder is donated to the vault. Since the curator controls the instant withdrawal fee, it can indirectly influence the amount of fees collected by the protocol in this specific flow. It could prevent the protocol from collecting fees from instant-withdrawal fee donations, although doing so would also harm the curator himself. 2. A curator could front-run an instant withdrawal to capture fees: If the fee collected during an instant withdrawal is small enough that the curator fee computed during <code>distributeDonationRewards</code> is rounded down to zero, the curator could front-run the instant withdrawal transaction and increase the fee so that at least a non-zero amount is captured during the donation flow.
Recommendations	Make sure the curator is a trusted party.
Comments / Resolution	Acknowledged.

Issue_20	Donation is possible to Vaults with $\geq$ V2
Severity	Informational
Description	The following check is existent during <code>distributeDonationRewards</code>: <pre>if (IMigratableEntity[vault].version() < VAULT_V2_VERSION) { revert NoDonationSupport(); }</pre> This essentially allows for Vaults with $> VAULT_V2_VERSION$ to experience donations, which may become problematic in the future.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_21	Per-caller-fee introduction can be bypassed
Severity	Informational
Description	The <code>distributeDonationRewards</code> function fetches fees based on the caller and falls back whenever there is no fee for the caller, to the default fee. This allows for ambiguous execution.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_22	Incorrect event emission parameter
Severity	Informational
Description	The event during <code>distributeDonationRewards</code> is as follows: <pre><i>emit DistributeDonationRewards(msg.sender, vault, distributionAmount);</i></pre> If we examine the <code>msg.sender</code> parameter, it points to: <pre><i>event DistributeDonationRewards(address indexed adapter, address indexed vault, uint256 amount);</i></pre> Inherently pointing to “adapter”. However, adapter is only true in the adapter control-flow context and incorrectly labeled if the vault is the caller or any user.
Recommendations	Consider changing this parameter within the interface.
Comments / Resolution	Resolved.

Issue_23	Protocol and curator fees may frequently round down in instant-withdrawal fee flows
Severity	Informational
Description	In instant-withdrawal fee flows, both protocol and curator fees are rounded down. This may be acceptable if such amounts are treated as donations and fee capture is not considered essential in this context. However, if most donations are expected to come from instant-withdrawal fees, the mechanism effectively applies two additional fees on top of an already charged fee. Depending on the token precision and the amount involved, these fees may round down to zero quite frequently.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

CuratorFees

The `CuratorFees` contract contains similar logic as previously existed within the `VaultSnapshotRewards` contract and has been introduced due to the outsourcing of this logic. The storage slot is the same as in the old code. This contract calculates and allocates the `curatorFee` and allows curators to claim it.

Please note that issues which have previously been acknowledged, will not be raised again.

Privileged Functions

- `claimCuratorFees`

Core Invariants:

INV 1: only the curator of a vault is allowed to claim fees